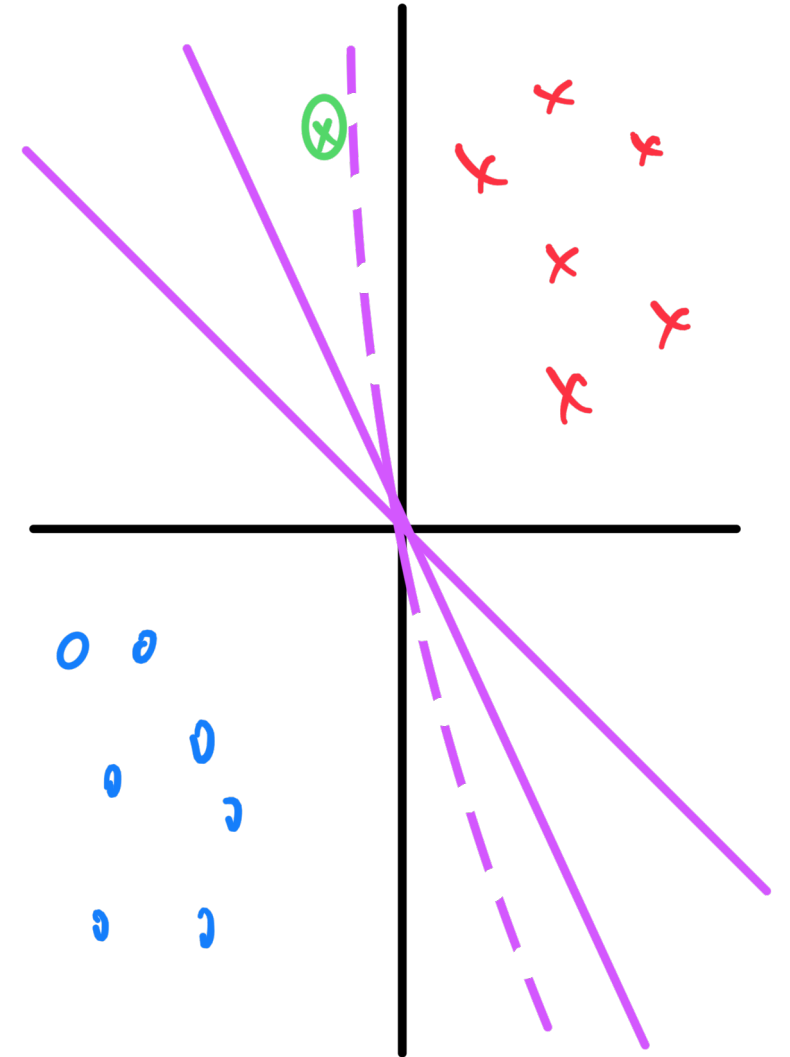


50.007 Machine Learning

Lecture 3

Hinge Loss

Recap



Training data

$$\mathcal{S}_n = \{ (x^{(i)}, y^{(i)}) \mid i = 1, \dots, n \}$$

- Features/Inputs $x^{(i)} = (x_1^{(i)}, \dots, x_d^{(i)})^\top \in \mathbb{R}^d$
- Labels/Output $y^{(i)} \in \{-1, +1\}$

Model

Set of *linear* classifiers $h: \mathbb{R}^d \rightarrow \{-1, +1\}$

$$h(x; \theta, \theta_0) = \text{sign}(\theta_d x_d + \cdots + \theta_1 x_1 + \theta_0)$$

$$= \text{sign}(\theta^\top x + \theta_0) = \begin{cases} +1 & \text{if } \theta^\top x + \theta_0 \geq 0, \\ -1 & \text{if } \theta^\top x + \theta_0 < 0. \end{cases}$$

Model Parameters

$$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$$



offset

Training Error

$$Loss(\theta, \theta_0; x, y) = \mathbb{I}[y \neq h(x; \theta, \theta_0)]$$

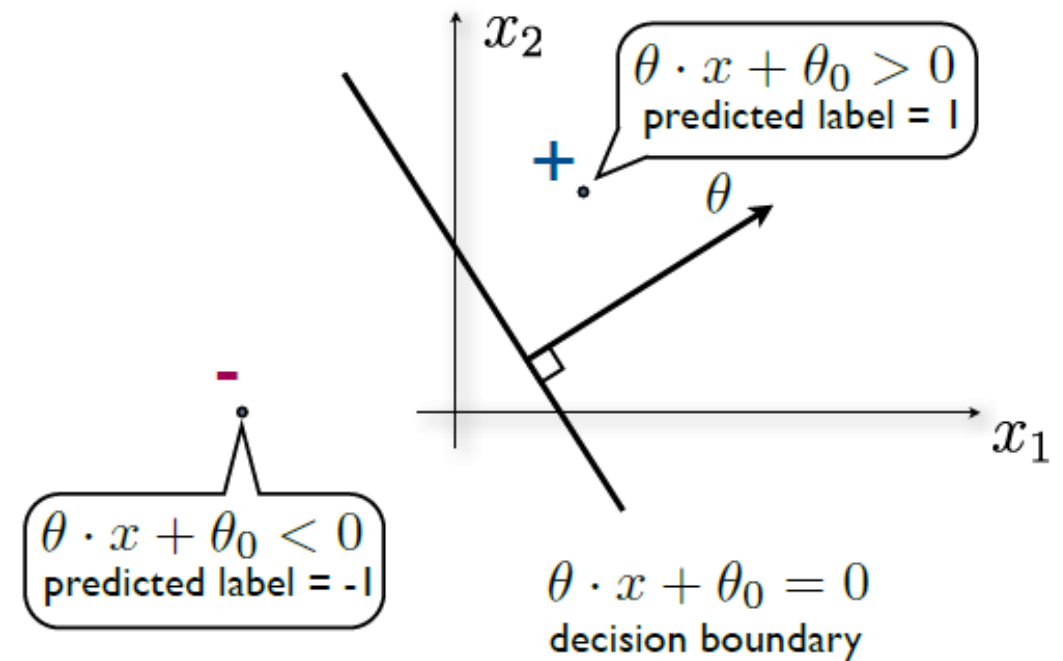
$$\mathcal{R}(\theta, \theta_0; \mathcal{S}_*) = \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_*} Loss(\theta, \theta_0; x, y)$$

$\mathbb{I}[\cdot]$ is the *indicator* function that returns a 1 if its argument is true, and 0 otherwise.

Decision Boundaries

Vector θ is orthogonal to the decision boundary.

Vector θ points in direction of region labelled +1.



Linear Classifier

- **Linear classifier (with offset):**

$$h(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0) = \begin{cases} +1 & \text{if } \theta^\top x + \theta_0 \geq 0, \\ -1 & \text{if } \theta^\top x + \theta_0 < 0. \end{cases}$$

- **Training error:**

$$\mathcal{E}_n(\hat{\theta}, \hat{\theta}_0) = \frac{1}{n} \sum_{t=1}^n \mathbb{I}[y^{(t)}(\hat{\theta} \cdot x^{(t)} + \hat{\theta}_0) \leq 0]$$

$\mathbb{I}[\cdot]$ is the *indicator* function that returns a 1 if its argument is true, and 0 otherwise.

- Linear classifier that achieves zero training error is called **realizable**.

+ve correctly classified
- 1 + misclassified

Zero-One Loss

$$\mathcal{E}_n(\hat{\theta}, \hat{\theta}_0) = \frac{1}{n} \sum_{t=1}^n \mathbb{I}[y^{(t)}(\hat{\theta} \cdot x^{(t)} + \hat{\theta}_0) \leq 0]$$



Let $\mathbb{I}[y^{(t)}(\hat{\theta} \cdot x^{(t)} + \hat{\theta}_0) \leq 0] = 1$ (0 otherwise) if

- $y \neq h(x; \theta)$, or
- (x, y) is on decision boundary

[misclassified]
[boundary]

Note that $y(\theta^\top x) \leq 0$ if

- $\theta^\top x$ and y differ in sign, or
- $\theta^\top x$ is zero

[misclassified]
[boundary]

Perceptron Algorithm

- Initialize the **weight** ($\theta = 0$).
- For each training example 't' in S_n , classify the instance
 - if correct, continue
 - else, $\theta^{(k+1)} = \theta^{(k)} + y^{(t)}x^{(t)}$
 $\theta_0^{(k+1)} = \theta_0^{(k)} + y^{(t)}$
- Terminate if the **training error** is zero (realizable) or a pre-determined number of iterations are completed (non-realizable).

Perceptron update rule

Linearly separable case

Theorem 2.1 *The perceptron update rule converges after a finite number of mistakes when the training examples are linearly separable through origin.*

- The above theorem implies, **zero training error** can be achieved using **perceptron update rule** for **linearly separable** training examples.

$$\mathcal{E}_n(\hat{\theta}, \hat{\theta}_0) = \frac{1}{n} \sum_{t=1}^n \mathbb{I}[y^{(t)}(\hat{\theta} \cdot x^{(t)} + \hat{\theta}_0) \leq 0] = 0$$

<http://www.cs.columbia.edu/~mcollins/courses/6998-2012/notes/perc.converge.pdf>

Perceptron Algorithm

1. Training Set (Linearly Separable)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$$

2. Model (Set of Perceptrons)

$$h(x; \theta) = \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d)$$

3. Training Error (Fraction of Misclassified/Boundary Points)

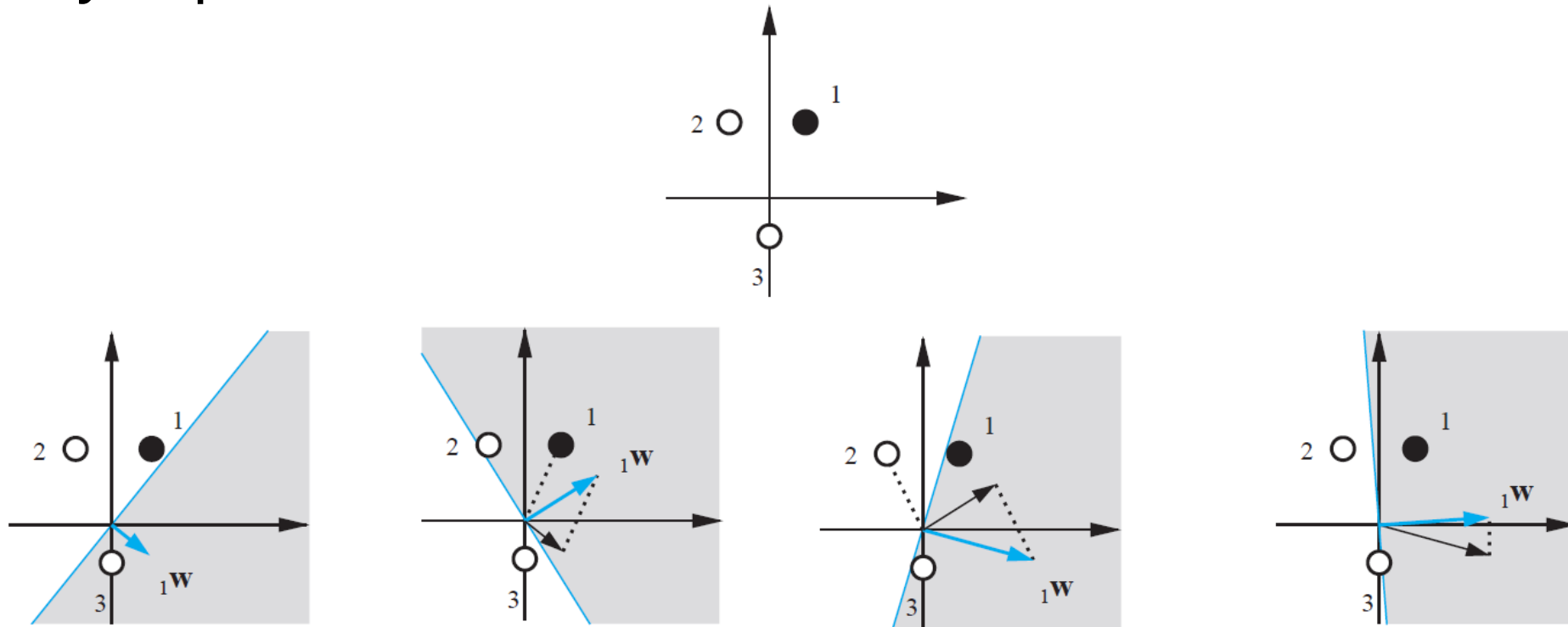
$$\epsilon_n(\theta) = \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_n} \mathbb{I} [y(\theta^\top x) \leq 0]$$

4. Algorithm (Mistake-Driven Algorithm)

$$\theta^{(k+1)} = \theta^{(k)} + y^{(t)} x^{(t)}$$

Example (Linearly Separable)

- Perceptron algorithm oscillates and terminates with zero error in linearly separable case.

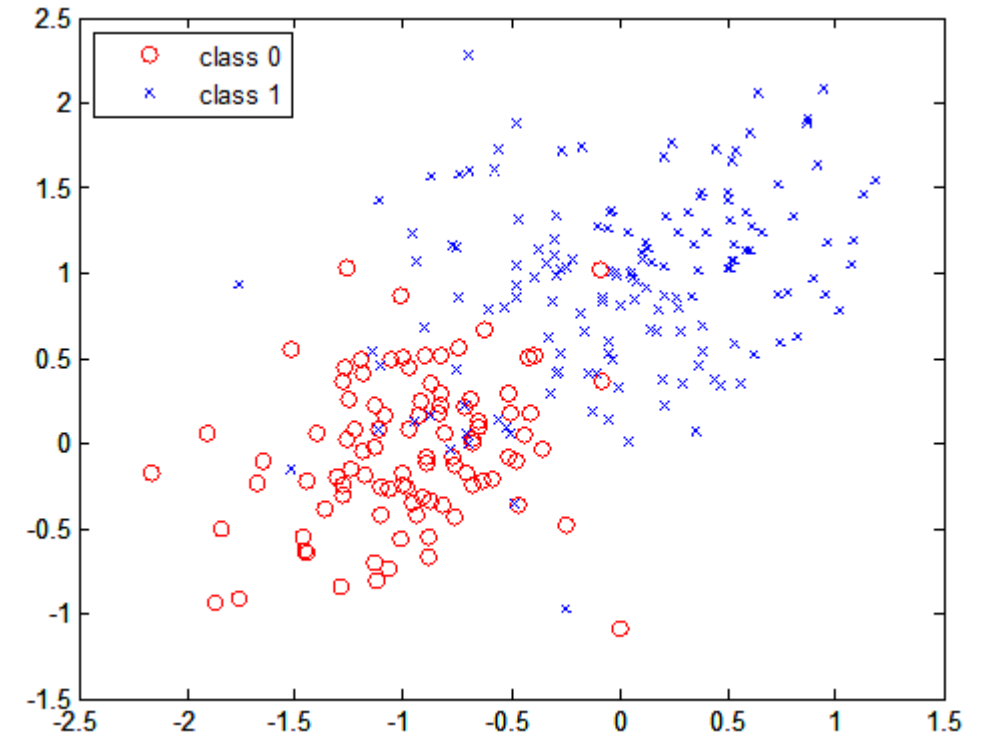


Linear Classifier

Non-Separable Case

Non-Separable case

- **Perceptron algorithm will not converge nor will it find the classifier with the smallest error, if training examples are linearly not separable.**
- **Goal: Find a classifier that minimizes the training error in the non-realizable case.**



Loss Functions

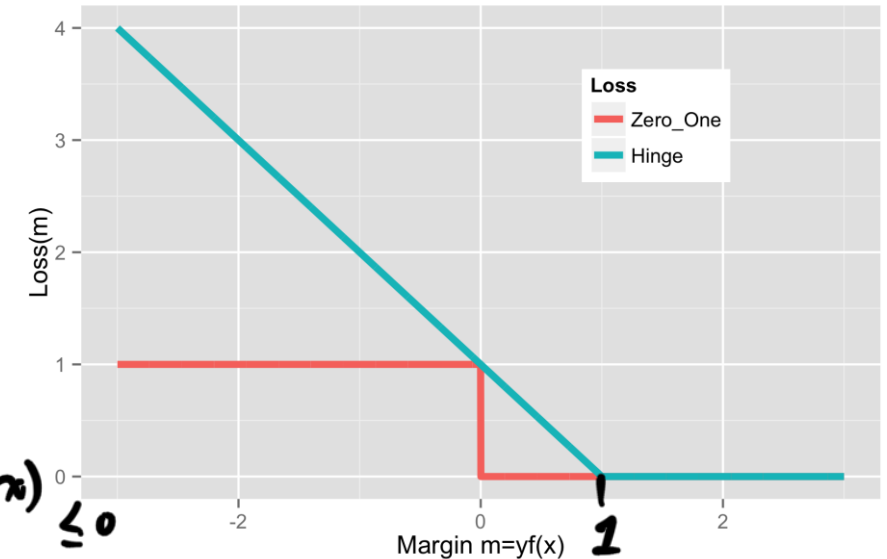
The penalty for hinge loss is $\propto$ for each prediction to the decision boundary

- Training Error / Empirical risk:

$$R_n(\theta) = \frac{1}{n} \sum_{\text{data } (x,y)} \text{Loss}(y(\theta^\top x))$$

- Zero-one Loss: $\text{Loss}_{0|1}(z) = \mathbb{I}[y(\theta^\top x) \leq 0]$

Return 1, if misclassified $\Rightarrow y(\theta^\top x) \leq 0$
0, otherwise $\Rightarrow y(\theta^\top x) > 0$



- Hinge Loss: $\text{Loss}_h(z) = \max\{1 - y(\theta^\top x), 0\}$

If $1 - y(\theta^\top x)$ is not too large,
e.g. $y(\theta^\top x) = 0.4$, $1 - 0.4 = 0.6$
the function will still penalize,
up till $y(\theta^\top x) > 1$ then stop.

If it is correctly classified,
we expect $y(\theta^\top x)$ to be $\uparrow$ true,
so $\text{Loss}_h(z)$ will be 0; zero loss.

CONVEX!

Penalize larger mistakes more.

Penalize near-mistakes, i.e. $0 \leq y(\theta^\top x) \leq 1$.

Hinge Loss (Examples)

$$h(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0) = \begin{cases} +1 & \text{if } \theta^\top x + \theta_0 \geq 0, \\ -1 & \text{if } \theta^\top x + \theta_0 < 0. \end{cases}$$

$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \text{Loss}_h(y^{(t)}(\theta \cdot x^{(t)})) = \frac{1}{n} \sum_{t=1}^n \max\{1 - y^{(t)}(\theta \cdot x^{(t)}), 0\}$$

Example 1

- original label = -1 and prediction score = 0.4 (this means the model predicted class as 1)
- penalty = $\max(0, 1 + 1(0.4)) = 1.4$ which is a very high penalty since the prediction was inaccurate

prediction score = $\theta \cdot x = 0.4$

Using Hinge loss function:

$$\text{Loss}_h(z) = \max\{1 - y(\theta^\top x), 0\}$$

$$y(\theta^\top x) = -1(0.4) = -0.4$$

$$\begin{aligned} \text{Loss}_h(z) &= \max\{1 - (-0.4), 0\} \\ &= \max\{1 + 0.4, 0\} \end{aligned}$$

$$\begin{aligned} \text{Loss}_h(z) &= \max\{1.4, 0\} \\ &= 1.4 \end{aligned}$$

$$\text{Loss}_{0.11}(z) = 1$$

Hinge Loss (Examples)

$$h(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0) = \begin{cases} +1 & \text{if } \theta^\top x + \theta_0 \geq 0, \\ -1 & \text{if } \theta^\top x + \theta_0 < 0. \end{cases}$$

$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \text{Loss}_h(y^{(t)}(\theta \cdot x^{(t)})) = \frac{1}{n} \sum_{t=1}^n \max\{1 - y^{(t)}(\theta \cdot x^{(t)}), 0\}$$

Example 1

- original label = -1 and prediction score = 0.4 (this means the model predicted class as 1)
- penalty = $\max(0, 1 + 1(0.4)) = 1.4$ which is a very high penalty since the prediction was inaccurate

Example 2

- original label = 1 and prediction score = (- 0.9) (this means the model predicted class as -1)
- penalty = $\max(0, 1 - 1(-0.9)) = 1.9$ which is a very high penalty since the prediction was inaccurate

Hinge Loss (Examples)

$$h(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0) = \begin{cases} +1 & \text{if } \theta^\top x + \theta_0 \geq 0, \\ -1 & \text{if } \theta^\top x + \theta_0 < 0. \end{cases}$$

$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \text{Loss}_h(y^{(t)}(\theta \cdot x^{(t)})) = \frac{1}{n} \sum_{t=1}^n \max\{1 - y^{(t)}(\theta \cdot x^{(t)}), 0\}$$

Example 1

- original label = -1 and prediction score = 0.4 (this means the model predicted class as 1)
- penalty = $\max(0, 1+1(0.4)) = 1.4$ which is a very high penalty since the prediction was inaccurate

Example 2

- original label = 1 and prediction score = (- 0.9) (this means the model predicted class as -1)
- penalty = $\max(0, 1-1(-0.9)) = 1.9$ which is a very high penalty since the prediction was inaccurate

Example 3

- original label = 1 and prediction score = 0.7 (this means the model predicted class as 1)
- penalty = $\max(0, 1-1(0.7)) = 0.3$ (loss is very less but not 0, since the prediction is accurate and has high confidence but not 100%)

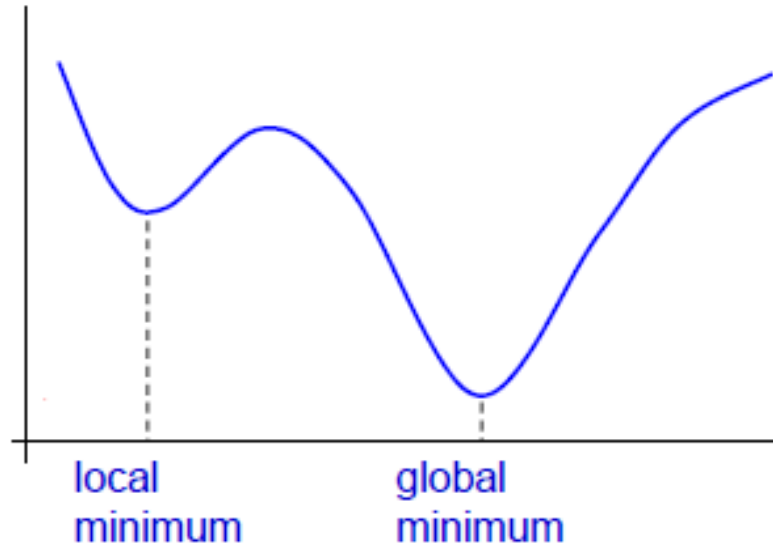
Hinge Loss

- Empirical risk using hinge loss:

$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \text{Loss}_h(y^{(t)}(\theta \cdot x^{(t)})) = \frac{1}{n} \sum_{t=1}^n \max\{1 - y^{(t)}(\theta \cdot x^{(t)}), 0\}$$

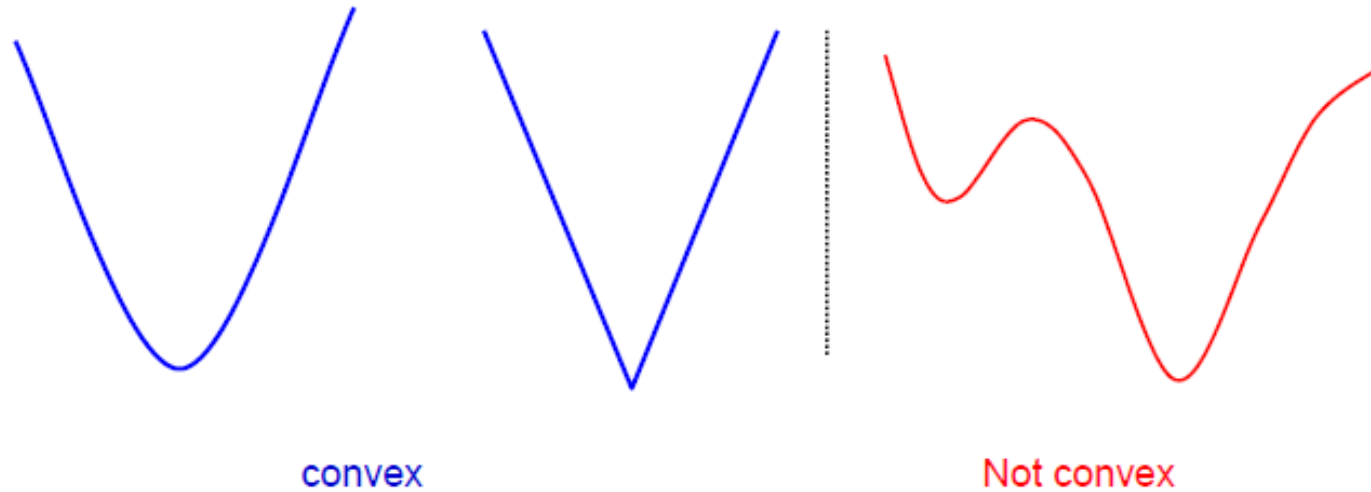
- **Convexity** of empirical risk allows us to find the **minimum** even in **non-realizable** case.

Optimization



- Does this loss function have a unique solution?
- If the loss function is convex, then a locally optimal point is globally optimal (provided the optimization is over a convex set, which it is in our case)

Convex Function Examples



A non-negative sum of convex functions is convex

Gradient Descent

- Use **gradient descent** to minimize $R_n(\theta)$

$$\nabla_{\theta} R_n(\theta) = \left[\frac{\partial R_n(\theta)}{\partial \theta_1}, \dots, \frac{\partial R_n(\theta)}{\partial \theta_d} \right]^T$$

- Positive gradient is in the direction where $R_n(\theta)$ **increases**.
- Need to update the weight in the **opposite direction** to minimize error.

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \nabla_{\theta} R_n(\theta)_{\theta=\theta^{(k)}}$$

Gradient Descent

$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \text{Loss}(y^{(t)}(\theta \cdot x^{(t)}))$$
$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \max\{1 - y^{(t)}(\theta \cdot x^{(t)}), 0\}$$

- Need to update the weight in the **opposite direction**.

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \nabla_{\theta} R_n(\theta)_{\theta=\theta^{(k)}}$$

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \nabla_{\theta} \text{Loss}_h(y^{(t)} \theta \cdot x^{(t)})_{\theta=\theta^{(k)}}$$

$$\nabla_{\theta} \text{Loss}_h(y^{(t)} \theta \cdot x^{(t)})_{\theta=\theta^{(k)}} = \nabla_{\theta} (1 - y^{(t)} \theta \cdot x^{(t)})_{\theta=\theta^{(k)}} = -y^{(t)} x^{(t)}$$

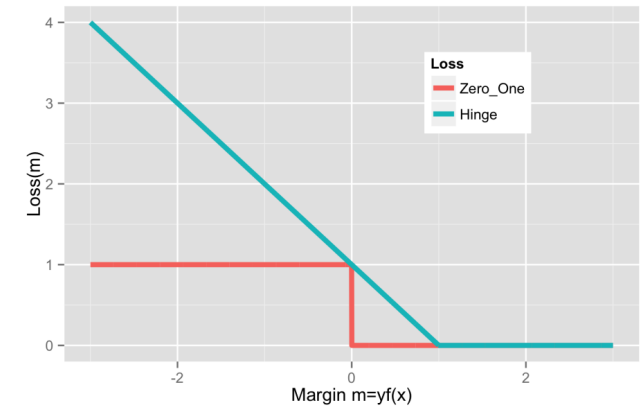
$$\theta^{(k+1)} = \theta^{(k)} + \eta_k y^{(t)} x^{(t)}$$

→ partial derivative w.r.t θ

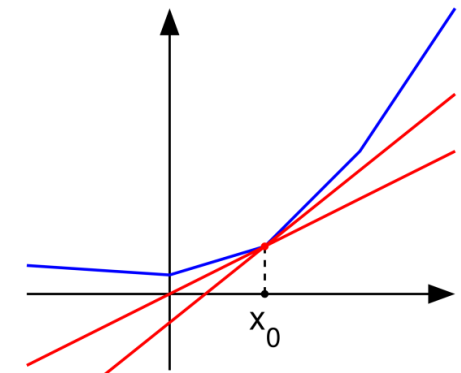
Gradient Descent

* note to take

- $R_n(\theta)$ is **not differentiable** everywhere as **hinge loss functions are piece-wise linear**.
- There are several **possible gradients** at the kinks which are collectively defined as **sub-differential**.
- To minimize $R_n(\theta)$, we need to select one possible gradient at any point.



Recall and observe the piece-wise linear shape of hinge loss function



A sample representation of sub-differential

Stochastic Gradient Descent

1. Initialize the **weight** $(\theta^{(0)} = 0)$.

2. Select $t \in \{1, \dots, n\}$ at random $\longrightarrow$ instead of sequential

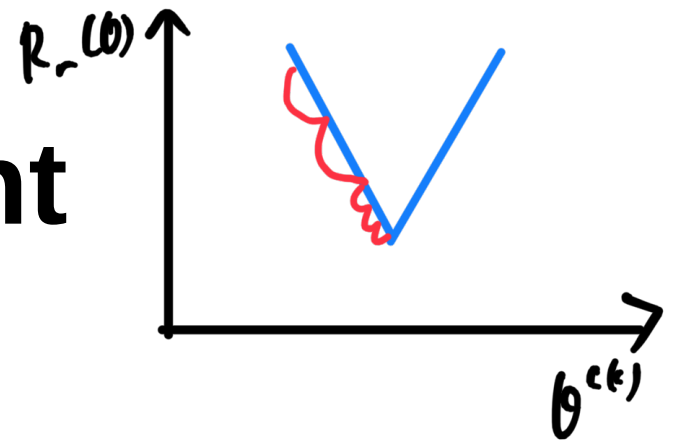
• If $y^{(t)}(\theta^{(k)} \cdot x^{(t)}) \leq 1$, then update the weight

$$\theta^{(k+1)} = \theta^{(k)} + \eta_k y^{(t)} x^{(t)}$$

$\longrightarrow$ include learning rate

penalize misclassification
marginally close to
decision boundary.
3. Repeat Step (2) until stopping criterion is met.
(e.g. when improvement in $R_n(\theta)$ is small enough)

Stochastic Gradient Descent



Differences from Perceptron algorithm:

- Near mistakes are also penalized

$$y^{(t)}(\theta^{(k)} \cdot x^{(t)}) \leq 1$$

As the value of $k \uparrow$, η_k should $\downarrow$ since it is inversely $\propto$ as we would want to prevent overshooting to find the min.

- Learning rate is decreasing (later updates will be smaller)

$$\eta_k = 1/(k + 1)$$

- Random samples avoid oscillation

Stochastic Gradient Descent

- Keep track of best solution (weight), $\theta^{(i_k)}$, where, $i_k = \operatorname{argmin}_{i=1,\dots,k} R_n(\theta^{(i)})$ is monotonically decreasing.
- **Note: asymptotically empirical risk does not become zero as the points may not be linearly separable.**

Hinge Loss SGD Algorithm

1. Training Set (Not Necessarily Linearly Separable)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$$

2. Model (Set of Perceptrons)

$$h(x; \theta) = \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d)$$

3. Training Loss (Hinge Loss)

$$R_n(\theta) = \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_n} \max\{1 - y(\theta^\top x), 0\}$$

4. Algorithm (Gradient Descent)

$$\theta^{(k+1)} = \theta^{(k)} + \eta_k y^{(t)} x^{(t)}$$

Summary

- **Linear Classification**

- Decision Boundary
- Linearly Separable

- **Perceptron Algorithm**

- Mistake-Driven
- Zero-One Loss
- Only for Linearly Separable Data

- **Hinge Loss**

- Gradient Descent
- Hinge Loss SGD Algorithm
- Differences with Perceptron Algorithm
- OK for Non Linearly Separable Data

Intended Learning

Hinge Loss

- Write down the hinge loss, and plot its graph. Write down the training loss and its gradient.
- List differences between the hinge loss SGD algorithm and the perceptron algorithm.
- Explain why the hinge loss SGD applies to non linearly separable data while the perceptron algorithm does not.